

Modern Object Detection Systems

1. Introduction

Object detection is the computer vision task of both locating and classifying objects within an image, producing a bounding box and a class label with a confidence score for each detected object. This differs from plain image classification, which assigns a single label to the whole image without localization. Over the past decade, deep learning has produced several distinct architectural families for this task, each with different trade-offs between inference speed, detection accuracy, and computational cost. This report discusses four influential approaches — YOLO, Faster R-CNN, SSD, and DETR — and compares them on speed, accuracy, and practical use cases.

2. Architectures

2.1 YOLO (You Only Look Once)

YOLO reframes object detection as a single regression problem: a single convolutional network looks at the whole image once and directly predicts bounding boxes and class probabilities in one forward pass. This single-stage design is what makes YOLO dramatically faster than earlier region-proposal-based methods. The architecture is generally described in three parts: a backbone (extracts feature maps at multiple scales, e.g. CSPDarknet in YOLOv8), a neck (fuses multi-scale features, e.g. PANet/FPN-style layers), and a detection head (predicts boxes, objectness/confidence, and class scores). Successive versions (YOLOv3 through YOLOv8 and beyond) have refined the backbone, added anchor-free detection heads, and improved training strategies, steadily closing the accuracy gap with two-stage detectors while retaining real-time speed.

2.2 Faster R-CNN

Faster R-CNN is a two-stage detector. The first stage, a Region Proposal Network (RPN), scans the feature map and proposes candidate object regions. The second stage takes those proposals, extracts fixed-size features via RoI pooling, and performs classification plus bounding-box regression on each proposal individually. Because it separates 'where might an object be' from 'what is it and exactly where are its edges,' Faster R-CNN typically achieves higher localization accuracy than single-stage detectors, particularly on small or overlapping objects — at the cost of running two networks in sequence, which is significantly slower than single-pass methods.

2.3 SSD (Single Shot MultiBox Detector)

SSD is also a single-stage detector, but structured differently from YOLO. It attaches detection heads to multiple feature-map layers at different depths of the backbone network, so it predicts boxes at several scales simultaneously (shallow layers catch small objects, deeper layers catch large objects) using a fixed set of default anchor boxes per location. This multi-scale prediction happens in one pass, making SSD fast, but its reliance on shallower feature maps for small-object detection makes it comparatively weaker than YOLO's later versions or Faster R-CNN at detecting small or densely packed objects.

2.4 DETR (Detection Transformer)

DETR takes a fundamentally different approach: it treats object detection as a direct set-prediction problem using a Transformer encoder-decoder, removing the need for hand-designed components like anchor boxes and non-maximum suppression entirely. A CNN backbone extracts features, a Transformer encoder models global relationships across the whole image via self-attention, and a decoder with a fixed set of learned 'object queries' directly outputs a fixed-size set of predictions matched to ground truth via bipartite matching during training. The global attention mechanism gives DETR strong contextual understanding, useful for cluttered or overlapping scenes,

but it is compute-heavy and historically has required long training schedules to converge, though later variants (e.g. Deformable DETR) address this.

3. Comparison — Speed, Accuracy, Use Cases

Model	Speed	Accuracy	Typical Use Cases
YOLO (v5–v8)	Very fast — single-pass, real-time (30–150+ FPS on GPU)	Good, improves with each version; slightly behind two-stage detectors on small/dense objects	Real-time video, mobile/edge apps, robotics, surveillance, gesture/PPE detection
Faster R-CNN	Slow — two-stage pipeline (5–20 FPS typical)	High — strong on small and overlapping objects	Medical imaging, satellite/aerial imagery, quality inspection where accuracy > speed
SSD	Fast — single-pass, faster than Faster R-CNN, slightly slower than YOLO	Moderate — weaker on small objects due to shallow early feature maps	Mobile and embedded vision, general-purpose real-time detection with moderate accuracy needs
DETR	Slow to moderate — transformer attention is compute-heavy; slow convergence in training	High — strong global context, excels at complex scenes and overlapping objects	Research, complex-scene understanding, scenarios where box-anchor tuning is undesirable

4. Discussion

The four architectures represent a clear trade-off spectrum. YOLO and SSD, as single-stage detectors, prioritize speed and are well suited to real-time and edge-deployment scenarios — this project's own hand gesture detection system uses YOLOv8n for exactly this reason, since live webcam inference demands low latency far more than it demands the last percentage point of mAP. Faster R-CNN sacrifices speed for localization precision and remains a strong choice when accuracy is the dominant requirement and inference can run offline or on stronger hardware. DETR represents a newer paradigm shift away from anchor-based design entirely, trading engineering complexity for a cleaner, attention-based formulation — at the cost of heavier compute and, in earlier versions, slower convergence.

In practice, the choice of architecture depends heavily on deployment constraints. Speed-critical, resource-constrained applications (mobile apps, robotics, live video) tend to favor YOLO or SSD. Accuracy-critical, less time-sensitive applications (medical imaging, industrial inspection) tend to favor Faster R-CNN. Research and complex-scene applications increasingly explore DETR-family models, especially as more efficient variants close the speed gap.

5. Conclusion

There is no single 'best' object detection architecture — each of YOLO, Faster R-CNN, SSD, and DETR occupies a different point on the speed-accuracy-complexity trade-off curve. Understanding these trade-offs, rather than defaulting to the most popular model, is what allows a practitioner to choose the right tool for a given problem's real-world constraints.